

From Linear Regression to GPT

슬라이드별 발표 해설 노트

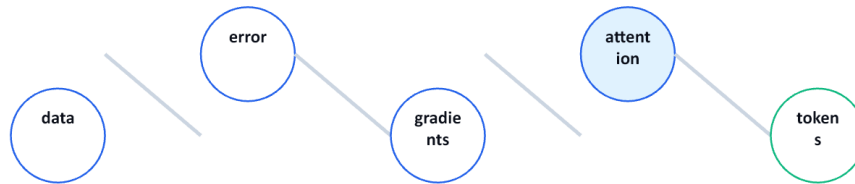
이 PDF는 영어 PPT를 발표할 때 사용할 한국어 설명 자료입니다. 각 페이지는 슬라이드 이미지와 핵심 설명으로 구성되어 있습니다.

권장 사용법: 슬라이드에는 짧은 영어 문구만 두고, 발표자는 이 노트를 바탕으로 개념의 배경과 연결 관계를 설명합니다.

Slide 01. From Linear Regression to GPT

From Linear Regression to GPT

A visual path from errors to attention to next-token prediction



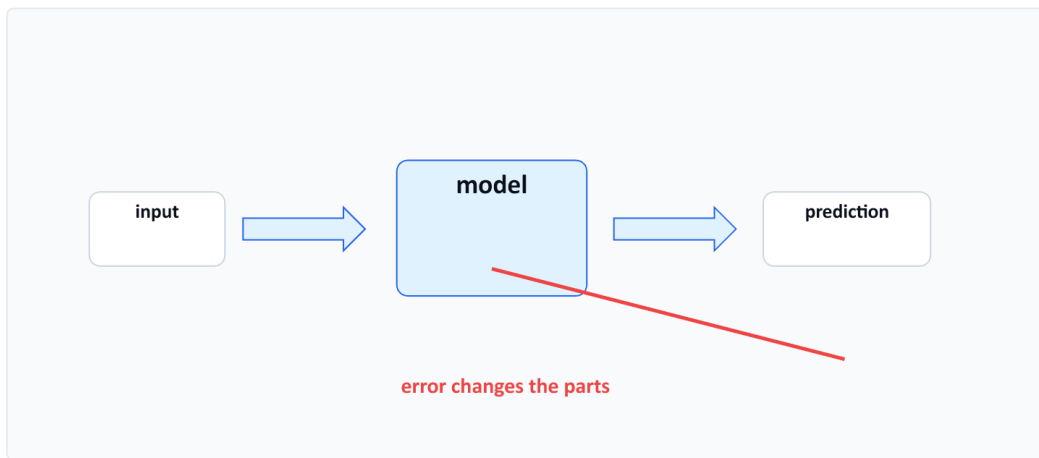
01

- 전체 여정의 지도를 보여주는 장입니다. 단순한 데이터와 오차에서 시작해, gradient, attention, token prediction으로 이어진다고 먼저 말합니다.
- 핵심 메시지는 GPT가 갑자기 등장한 기술이 아니라, 모델이 오차를 줄이며 패턴을 학습한다는 아이디어가 매우 크게 확장된 결과라는 점입니다.
- 청중에게 수식이 나와도 겁내지 않아도 된다고 안내합니다. 각 수식은 계산 자체보다 왜 필요한지를 설명하기 위한 도구입니다.

Slide 02. A model is a function with adjustable parts

Machine learning

A model is a function with adjustable parts



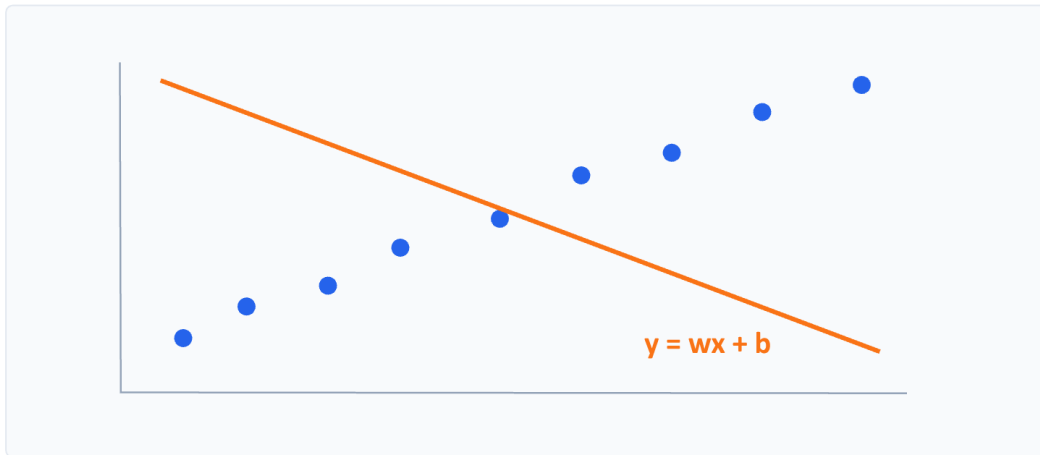
02

- 머신러닝 모델을 '조절 가능한 부분을 가진 함수'로 정의합니다. 입력을 넣으면 예측이 나오고, 예측이 틀리면 그 오차가 모델을 바꿉니다.
- 여기서 조절 가능한 부분이 weight와 bias입니다. GPT처럼 거대한 모델도 결국 엄청나게 많은 조절 가능한 숫자를 가진 함수입니다.
- 고정된 규칙을 사람이 모두 쓰는 전통적 프로그램과 달리, 머신러닝은 데이터로부터 이 숫자들을 찾아냅니다.

Slide 03. Linear regression fits a line

First model

Linear regression fits a line



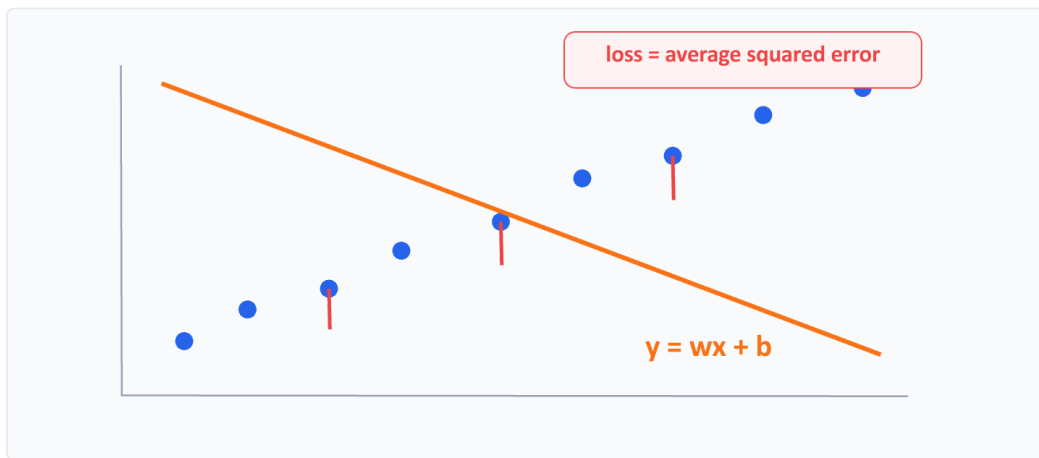
03

- 선형 회귀는 가장 단순한 학습 모델입니다. 점들은 실제 데이터이고, 주황색 선은 모델의 예측 규칙입니다.
- 수식 $y = wx + b$ 에서 w 는 기울기, b 는 시작 위치입니다. 이 두 숫자를 잘 조절하면 데이터에 가까운 선을 만들 수 있습니다.
- 이 장의 목적은 '학습이란 적절한 숫자를 찾는 과정'이라는 감각을 만드는 것입니다.

Slide 04. Loss gives wrongness a number

Measure

Loss gives wrongness a number



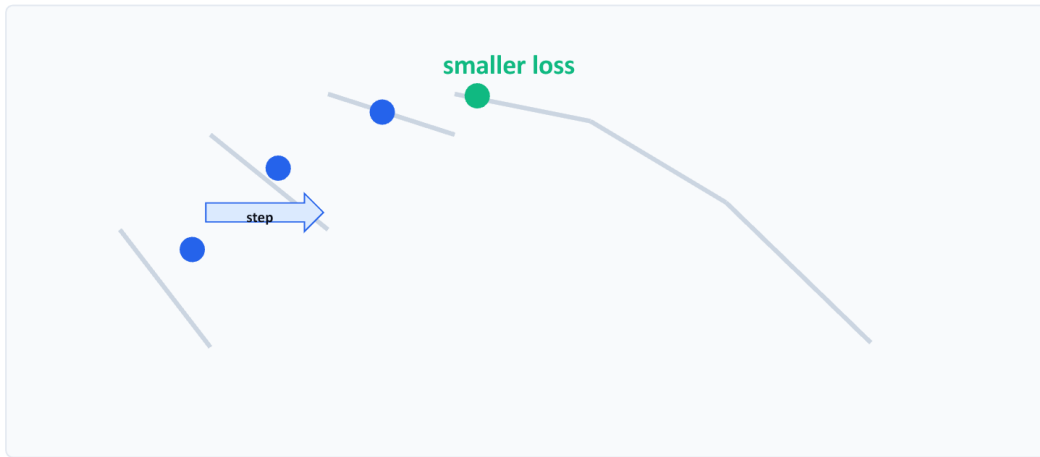
04

- 모델이 얼마나 틀렸는지를 숫자로 만든 것이 loss입니다. 빨간 선은 예측과 실제값 사이의 차이를 뜻합니다.
- 평균 제곱 오차는 차이를 제곱해서 평균낸 값입니다. 크게 틀린 예측에 더 큰 벌점을 주기 때문에 직관적으로 이해하기 좋습니다.
- 학습의 목표는 이 loss를 작게 만드는 것입니다. 즉 모델은 정답을 직접 외우는 것이 아니라 wrongness를 줄입니다.

Slide 05. Gradient descent walks downhill

Learning

Gradient descent walks downhill



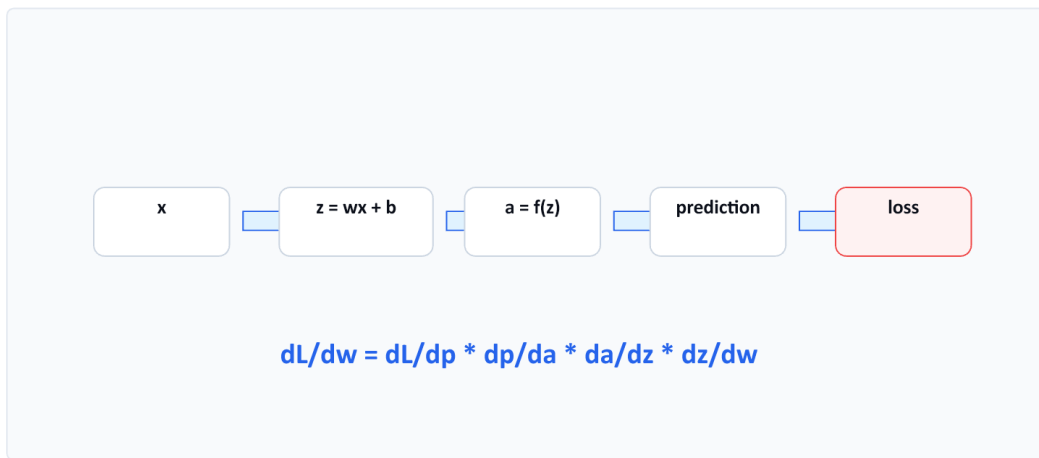
05

- Gradient descent는 loss가 작아지는 방향으로 조금씩 이동하는 방법입니다. 산 위에서 아래쪽을 찾아 내려가는 그림으로 설명합니다.
- 각 점은 현재 모델의 상태입니다. 기울기를 보고 어느 방향으로 움직이면 loss가 줄어드는지 판단합니다.
- 한 번에 완벽한 답을 찾는 것이 아니라, 작은 step을 여러 번 반복한다는 점을 강조합니다.

Slide 06. The chain rule connects causes

Calculus

The chain rule connects causes



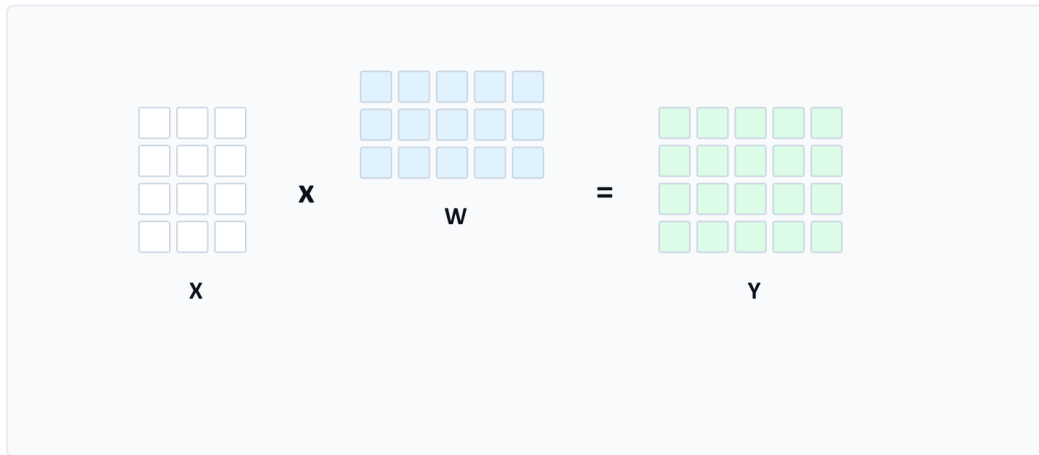
06

- 신경망은 여러 계산이 이어진 구조입니다. 입력이 중간 계산을 만들고, 중간 계산이 예측을 만들고, 예측이 loss를 만듭니다.
- Chain rule은 loss가 맨 앞의 weight에 얼마나 영향을 받는지 계산하게 해줍니다.
- 이 장에서는 수식을 외우게 하기보다, 뒤쪽의 오차가 앞쪽의 원인까지 연결된다는 직관을 전달합니다.

Slide 07. Matrices move many numbers at once

Parallel math

Matrices move many numbers at once



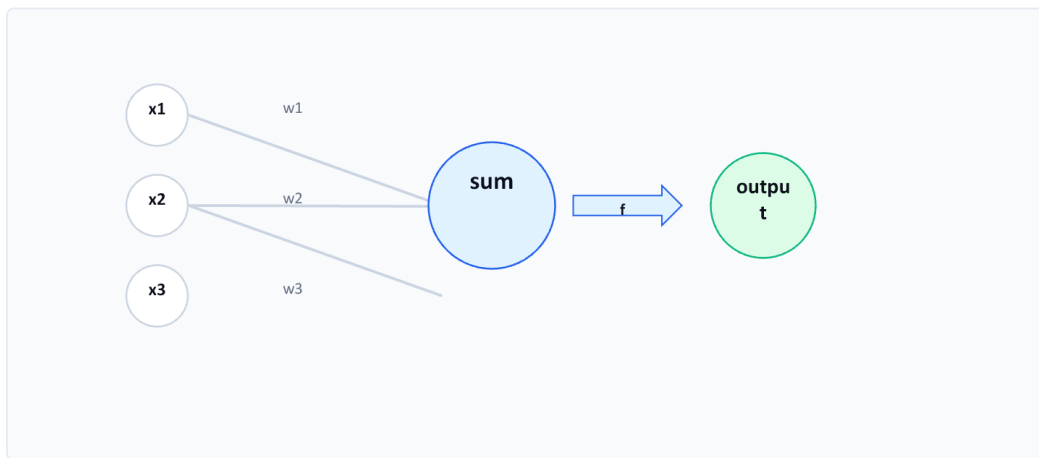
07

- 행렬곱은 많은 입력과 많은 weight를 한 번에 계산하기 위한 표현입니다. 하나의 뉴런이 아니라 수많은 뉴런을 동시에 계산한다고 설명합니다.
- X는 데이터 묶음, W는 weight 묶음, Y는 결과 묶음입니다. 딥러닝은 사실상 이런 행렬 연산을 매우 많이 반복합니다.
- GPU가 중요한 이유도 여기서 연결됩니다. GPU는 큰 행렬 계산을 병렬로 빠르게 처리하는 데 강합니다.

Slide 08. A neuron is a tiny calculator

Neural networks

A neuron is a tiny calculator



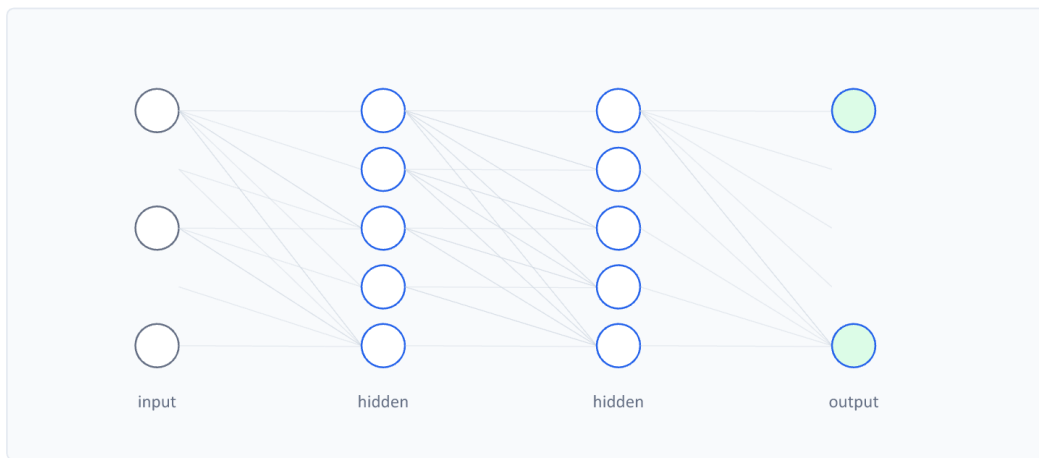
08

- 뉴런 하나는 여러 입력에 weight를 곱하고 더한 뒤, activation function을 통과시키는 작은 계산기입니다.
- 입력 x_1 , x_2 , x_3 가 각각 w_1 , w_2 , w_3 와 곱해지고 합쳐진다는 구조를 설명합니다.
- activation function은 단순한 직선 계산만 반복하지 않도록 비선형성을 추가합니다.

Slide 09. A network stacks many neurons

Depth

A network stacks many neurons



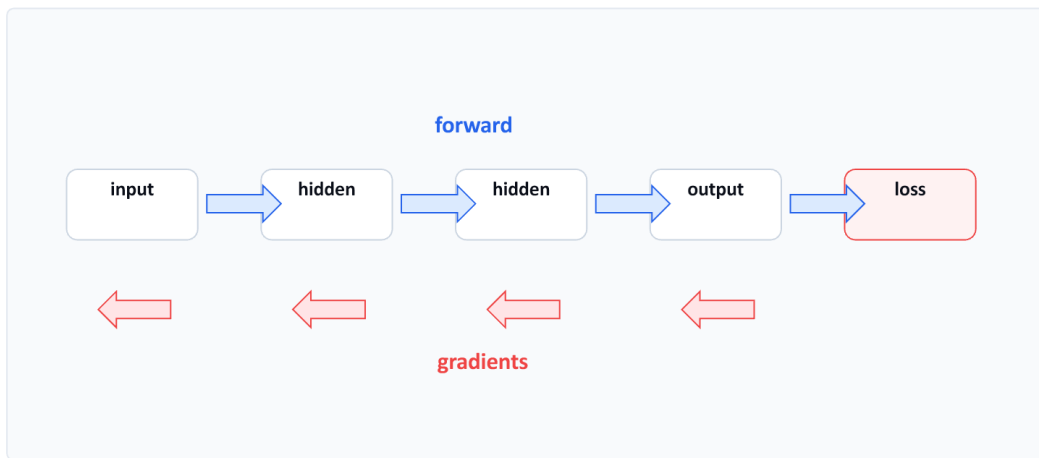
09

- 신경망은 이런 작은 뉴런을 층으로 쌓은 구조입니다. 앞 층의 출력이 다음 층의 입력이 됩니다.
- 층이 깊어질수록 단순한 패턴에서 복잡한 패턴으로 조합될 수 있습니다.
- 여기서 중요한 말은 depth입니다. GPT도 기본 아이디어는 층을 많이 쌓은 네트워크입니다.

Slide 10. Backprop sends error backward

Training

Backprop sends error backward



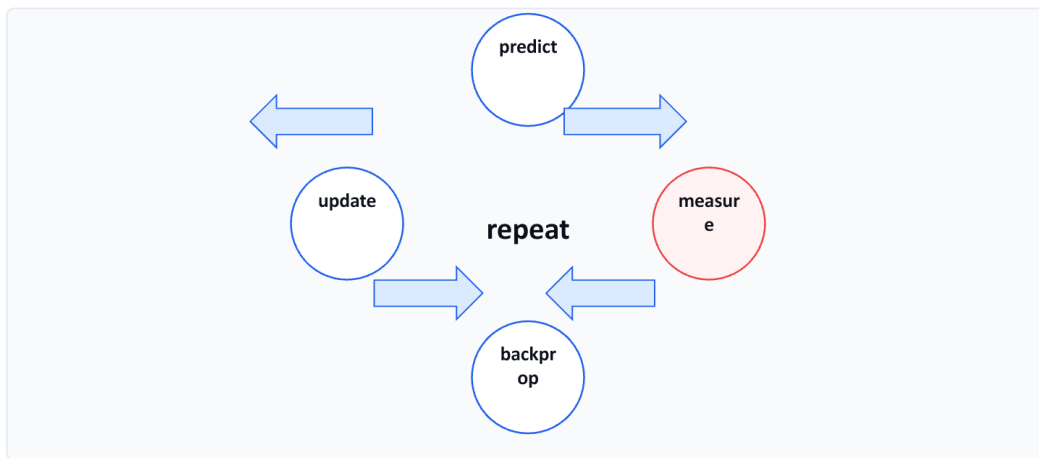
10

- Forward pass에서는 입력에서 loss까지 계산이 앞으로 진행됩니다. Backpropagation에서는 loss에서 시작해 gradient가 뒤로 전달됩니다.
- 각 weight는 '내가 오차에 얼마나 기여했는가'에 대한 신호를 받습니다.
- 그 신호를 이용해 weight를 조금씩 수정하면 다음 예측이 더 좋아질 가능성이 생깁니다.

Slide 11. Training is a loop

Loop

Training is a loop



11

- 학습은 predict, measure, backprop, update의 반복입니다. 이 루프가 수천, 수백만, 수십억 번 반복될 수 있습니다.
- 모델은 한 번의 설명을 듣고 배우는 것이 아니라, 많은 예시에서 오차를 보고 숫자를 계속 조정합니다.
- 이 루프는 선형 회귀부터 GPT까지 공통으로 유지되는 핵심 구조입니다.

Slide 12. Words become vectors

Embeddings

Words become vectors



12

- 컴퓨터는 단어를 그대로 이해하지 못하므로 숫자 벡터로 바꿔야 합니다. 이 숫자 표현을 embedding이라고 부릅니다.
- 벡터는 단어의 의미나 쓰임을 담은 좌표처럼 생각할 수 있습니다.
- 이 장은 자연어가 딥러닝에 들어가기 위해 먼저 숫자로 바뀐다는 점을 설명합니다.

Slide 13. Meaning becomes geometry

Embeddings

Meaning becomes geometry



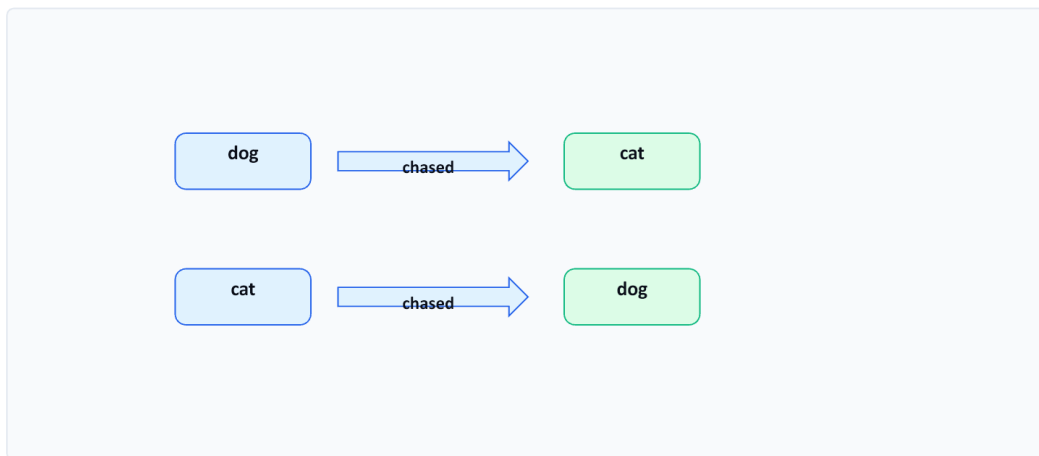
13

- Embedding 공간에서는 비슷한 의미의 단어가 가까운 위치에 놓일 수 있습니다.
- king과 queen, cat과 dog가 가까운 예시를 통해 의미가 숫자 공간의 거리와 방향으로 표현될 수 있음을 보여줍니다.
- 실제 모델의 공간은 2차원이 아니라 수백 또는 수천 차원이지만, 직관을 위해 2차원으로 그린 것입니다.

Slide 14. Order changes the meaning

Sequences

Order changes the meaning



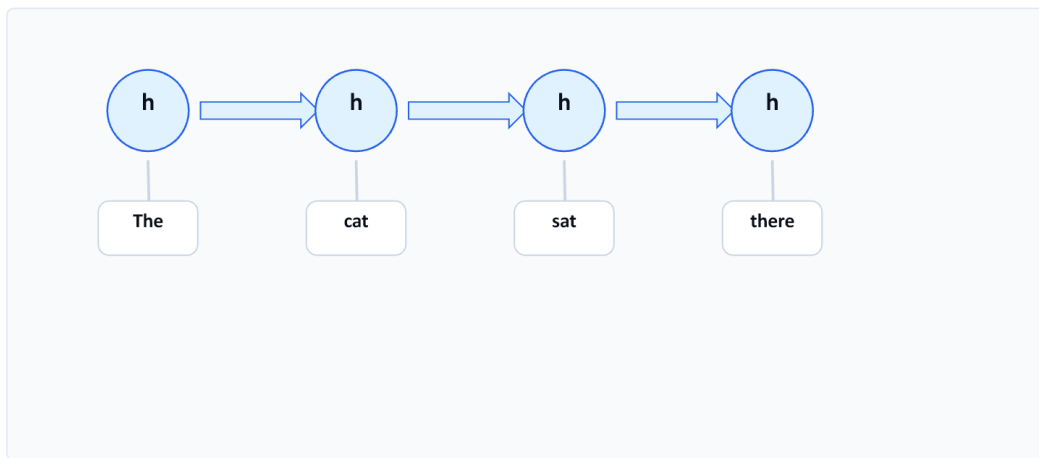
14

- 같은 단어라도 순서가 바뀌면 뜻이 바뀝니다. dog chased cat과 cat chased dog는 단어 목록은 비슷하지만 의미는 다릅니다.
- 따라서 언어 모델은 단어의 의미뿐 아니라 순서와 관계를 함께 처리해야 합니다.
- 이 문제가 sequence model이 필요한 이유입니다.

Slide 15. RNNs read one token at a time

Before attention

RNNs read one token at a time



15

- RNN은 문장을 왼쪽에서 오른쪽으로 한 토큰씩 읽습니다. hidden state가 이전 정보를 담는 메모리 역할을 합니다.
- 이 구조는 순서를 자연스럽게 처리할 수 있다는 장점이 있습니다.
- 하지만 모든 정보를 하나의 흐르는 상태에 계속 압축해야 하므로 긴 문장에서 어려움이 생깁니다.

Slide 16. Long memory fades

Limit

Long memory fades



16

- 문장이 길어지면 앞부분의 중요한 정보가 뒤까지 잘 전달되지 않을 수 있습니다.
- 초기 RNN 계열 모델은 장거리 의존성을 다루기 어렵고, 순차 처리 때문에 병렬화도 제한됩니다.
- 이 한계가 attention이 등장하는 중요한 배경입니다.

Slide 17. Attention lets words look around

Attention

Attention lets words look around



17

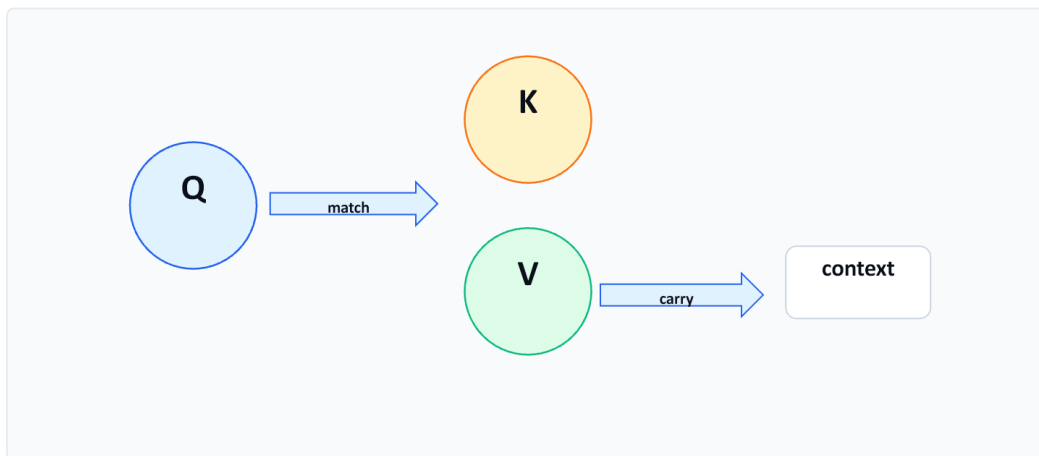
- Attention은 한 토큰이 다른 모든 토큰을 직접 바라보며 어떤 단어가 중요한지 판단하게 합니다.
- 예를 들어 cat이라는 단어를 이해할 때 주변 단어와 직접 비교할 수 있습니다.
- 핵심 변화는 '순서대로 기억하기'에서 '필요한 토큰을 직접 참조하기'로 바뀐 것입니다.

출처 참고: Transformer 관련 내용은 Vaswani et al. (2017), GPT 관련 내용은 OpenAI GPT 계열 논문 및 Brown et al. (2020)에 기반합니다.

Slide 18. Query, key, value

Mechanism

Query, key, value



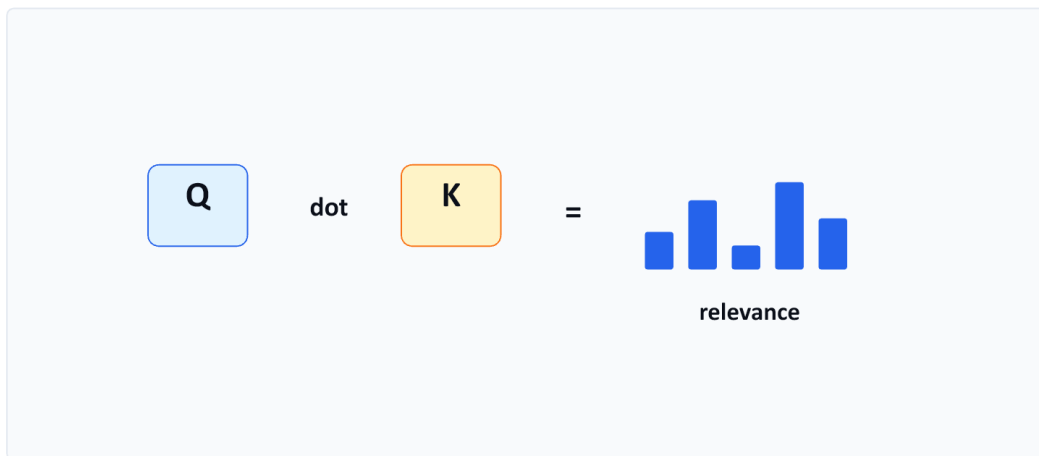
18

- Attention은 query, key, value라는 세 역할로 설명할 수 있습니다.
- Query는 내가 찾고 싶은 정보, key는 다른 토큰이 가진 검색 표지, value는 실제로 가져올 내용입니다.
- Query와 key를 비교해 관련성을 계산하고, 그 관련성에 따라 value를 섞어 context를 만듭니다.

Slide 19. Scores come from dot products

Mechanism

Scores come from dot products



19

- Query와 key의 dot product는 두 벡터가 얼마나 비슷한 방향을 보는지 측정합니다.
- 점수가 높으면 해당 토큰이 현재 토큰을 이해하는 데 더 중요하다는 뜻입니다.
- 이 점수들은 아직 확률이 아니라 raw score입니다. 다음 장에서 softmax로 변환됩니다.

Slide 20. Softmax makes weights

Mechanism

Softmax makes weights



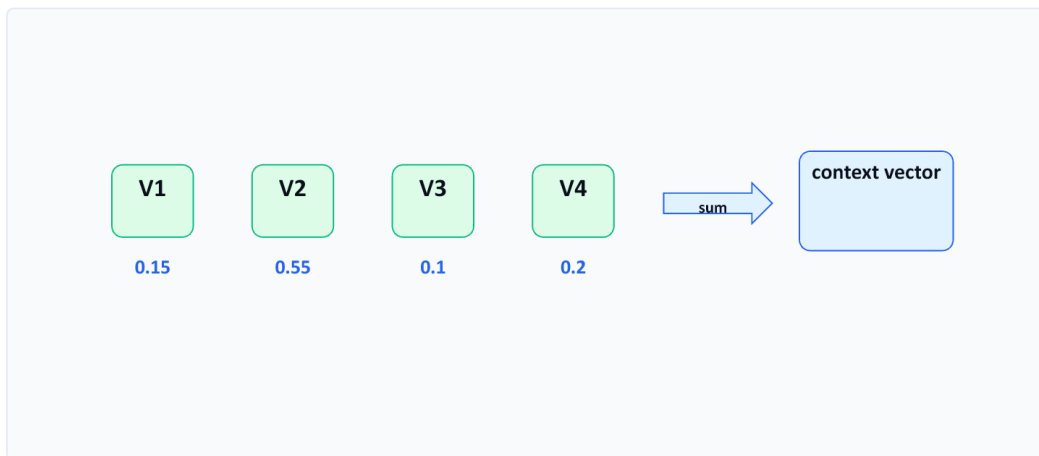
20

- Softmax는 raw score를 0과 1 사이의 가중치로 바꾸고, 전체 합이 1이 되게 만듭니다.
- 이렇게 하면 어떤 토큰을 얼마나 참고할지 비율로 해석할 수 있습니다.
- 높은 점수는 큰 attention weight가 되고, 낮은 점수는 작은 weight가 됩니다.

Slide 21. Values are mixed into context

Mechanism

Values are mixed into context



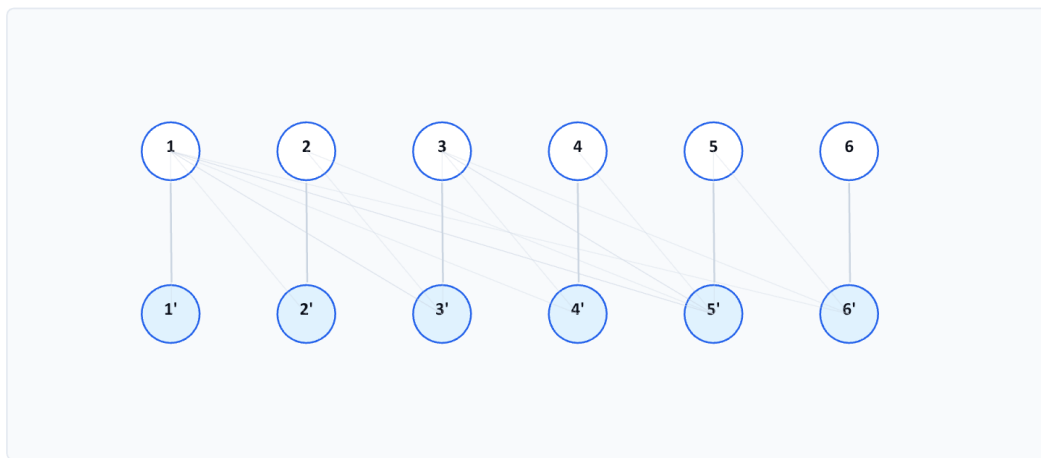
21

- Attention weight가 정해지면 value 벡터들을 가중합으로 섞습니다.
- 중요한 토큰의 value는 많이 반영되고, 덜 중요한 토큰의 value는 조금만 반영됩니다.
- 그 결과 현재 토큰은 문맥을 반영한 새로운 벡터가 됩니다.

Slide 22. Self-attention updates every token

Self-attention

Self-attention updates every token



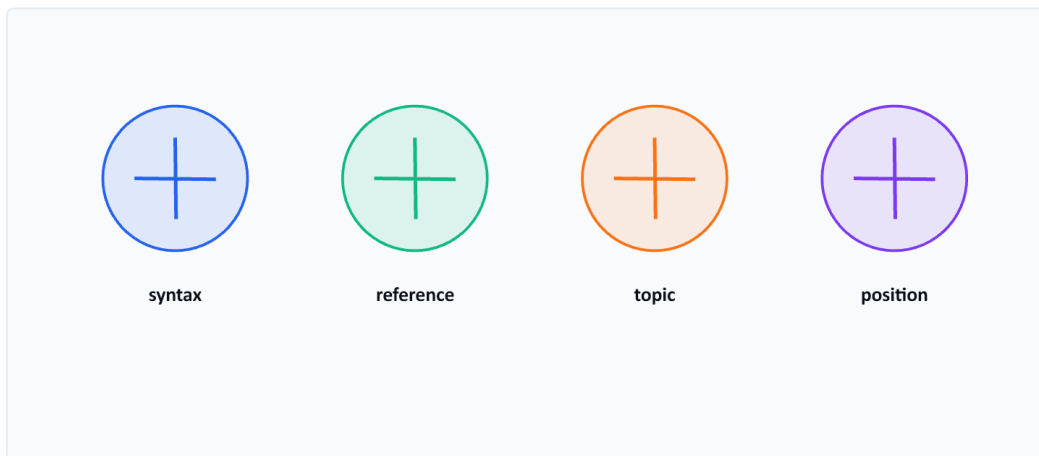
22

- Self-attention에서는 한 문장 안의 모든 토큰이 서로를 참고합니다.
- 각 토큰은 자기 자신뿐 아니라 다른 토큰들과의 관계를 반영해 업데이트됩니다.
- 이 덕분에 문장 전체의 관계를 한 층 안에서 넓게 반영할 수 있습니다.

Slide 23. Many heads learn different links

Multi-head

Many heads learn different links



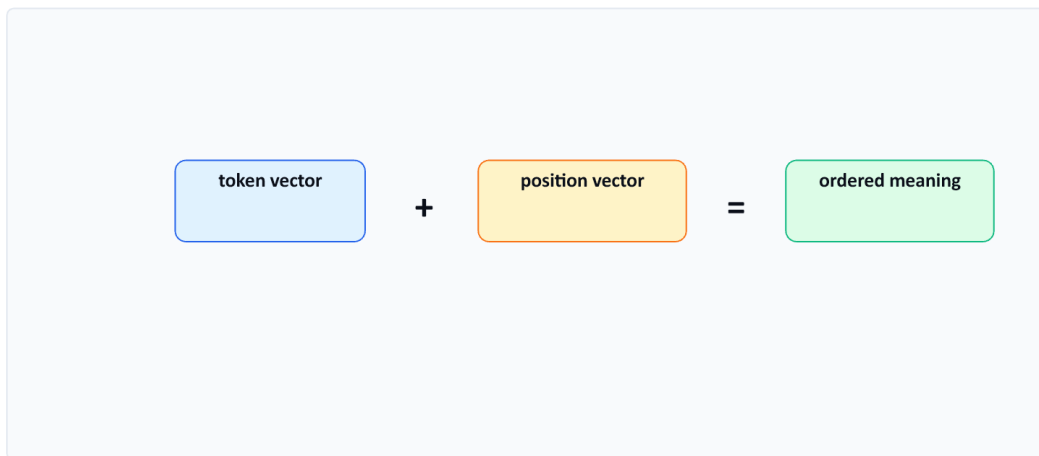
23

- Multi-head attention은 attention을 여러 별 병렬로 수행하는 구조입니다.
- 각 head는 서로 다른 관계를 배울 수 있습니다. 예를 들어 문법, 지시 대상, 주제, 위치 같은 관계입니다.
- 여러 관점을 합치면 한 가지 attention보다 더 풍부한 문맥 표현을 만들 수 있습니다.

Slide 24. Position is added to meaning

Order

Position is added to meaning



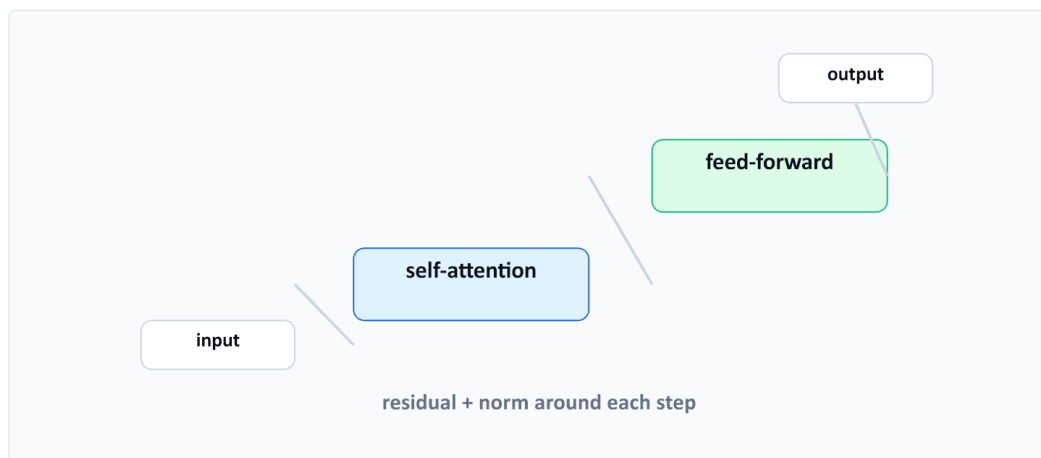
24

- Self-attention은 기본적으로 모든 토큰을 동시에 비교하므로 순서 정보를 따로 넣어줘야 합니다.
- 그래서 token vector에 position vector를 더합니다.
- 이렇게 하면 모델은 단어의 의미와 위치를 함께 가진 ordered meaning을 사용하게 됩니다.

Slide 25. A Transformer block mixes and computes

Block

A Transformer block mixes and computes



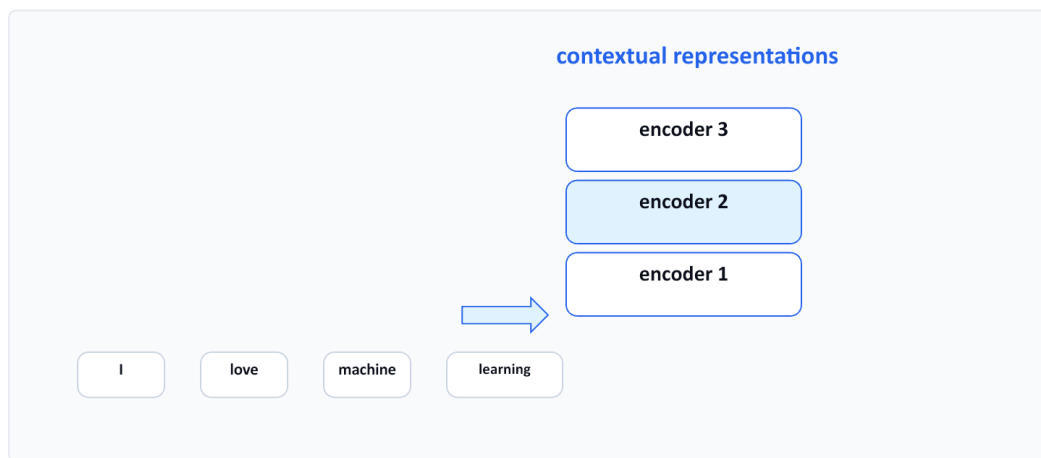
25

- Transformer block은 self-attention과 feed-forward network를 핵심 구성으로 갖습니다.
- Self-attention은 토큰들 사이의 정보를 섞고, feed-forward는 각 토큰의 표현을 더 계산합니다.
- Residual connection과 normalization은 깊은 네트워크가 안정적으로 학습되도록 돕습니다.

Slide 26. The encoder reads the whole input

Encoder

The encoder reads the whole input



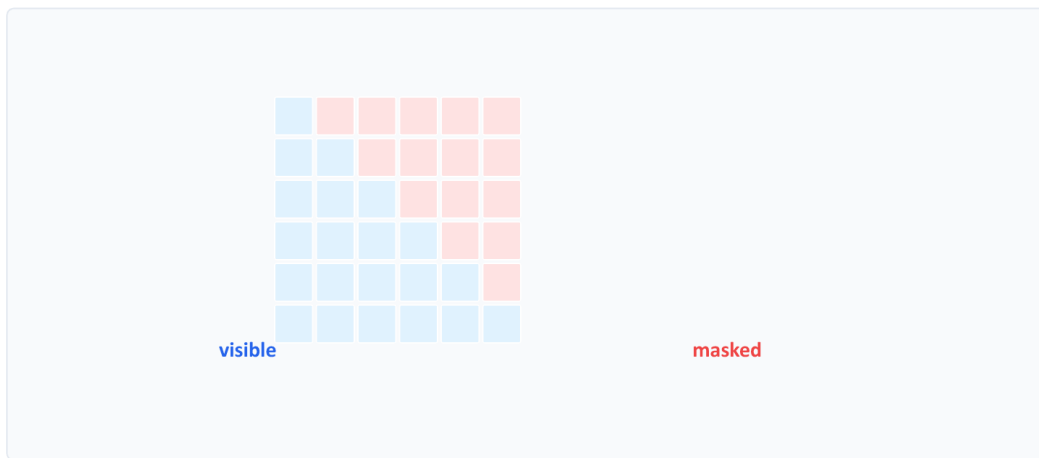
26

- Encoder는 입력 문장 전체를 읽고 각 토큰을 문맥이 반영된 표현으로 바꿉니다.
- 기계번역 같은 원래 Transformer 구조에서는 encoder가 source sentence를 이해하는 역할을 합니다.
- 여기서는 모든 입력 토큰을 함께 볼 수 있다는 점이 중요합니다.

Slide 27. The decoder hides the future

Decoder

The decoder hides the future



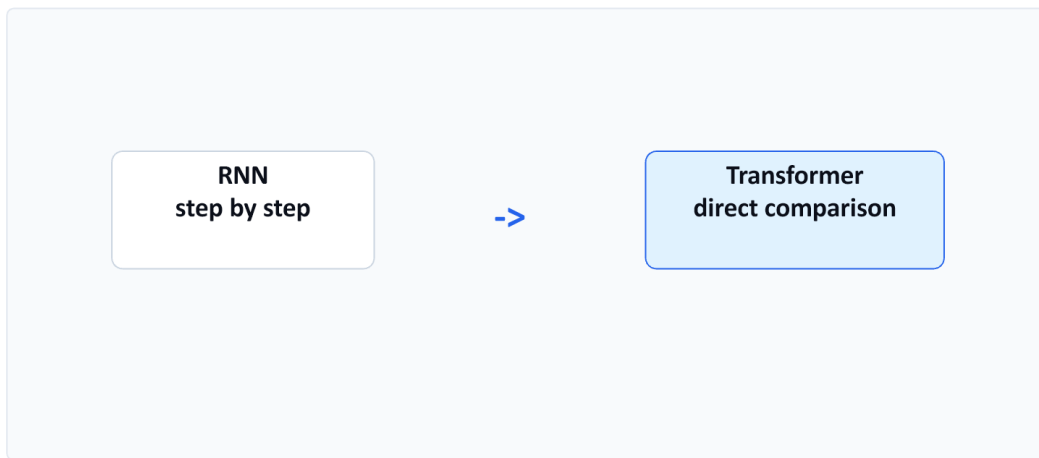
27

- Decoder는 문장을 생성해야 하므로 미래 토큰을 보면 안 됩니다.
- Masked self-attention은 현재 위치에서 이전 토큰만 볼 수 있게 제한합니다.
- 이 제약 덕분에 모델은 실제 생성 상황처럼 다음 토큰을 예측하도록 훈련됩니다.

Slide 28. Attention replaced recurrence

2017

Attention replaced recurrence



28

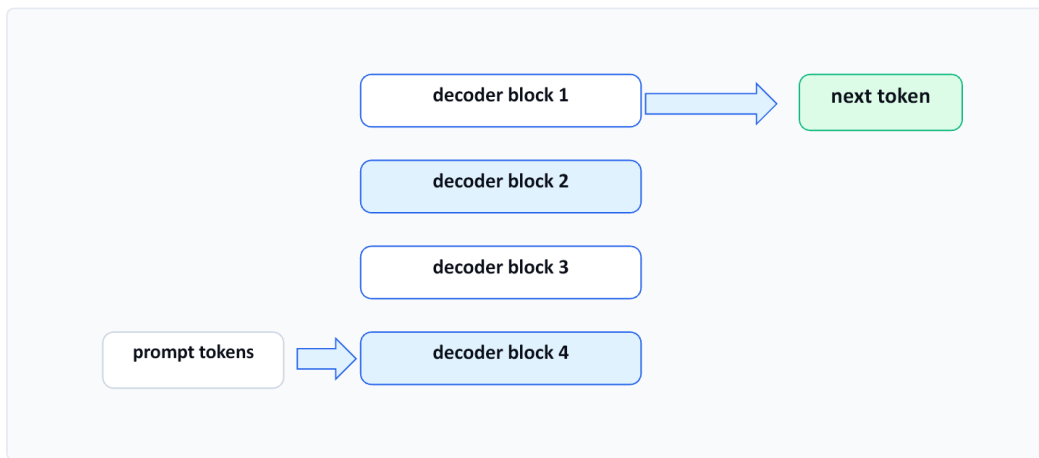
- Attention Is All You Need의 큰 메시지는 recurrence 없이 attention만으로 강력한 sequence model을 만들 수 있다는 것입니다.
- RNN은 순차적으로 읽지만, Transformer는 토큰 간 직접 비교를 중심으로 작동합니다.
- 이 변화는 병렬 처리와 확장성에서 큰 장점을 만들었습니다.

출처 참고: Transformer 관련 내용은 Vaswani et al. (2017), GPT 관련 내용은 OpenAI GPT 계열 논문 및 Brown et al. (2020)에 기반합니다.

Slide 29. GPT keeps the decoder idea

GPT

GPT keeps the decoder idea



29

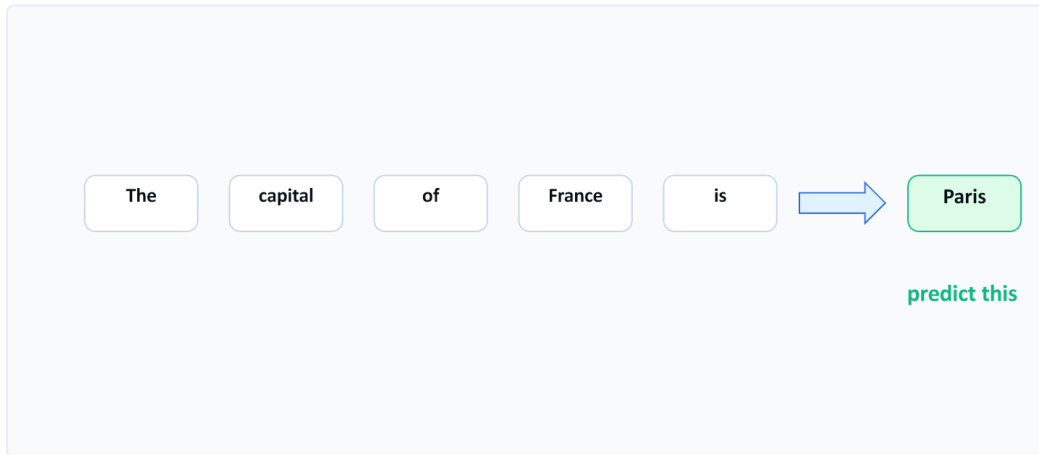
- GPT는 Transformer의 decoder 계열 아이디어를 사용해 prompt 다음의 token을 예측합니다.
- 여러 decoder block을 쌓아 매우 깊은 모델을 만들고, masked self-attention으로 왼쪽 문맥만 봅니다.
- GPT의 G는 generative, P는 pre-trained, T는 Transformer를 뜻합니다.

출처 참고: Transformer 관련 내용은 Vaswani et al. (2017), GPT 관련 내용은 OpenAI GPT 계열 논문 및 Brown et al. (2020)에 기반합니다.

Slide 30. Training target: the next token

Objective

Training target: the next token



30

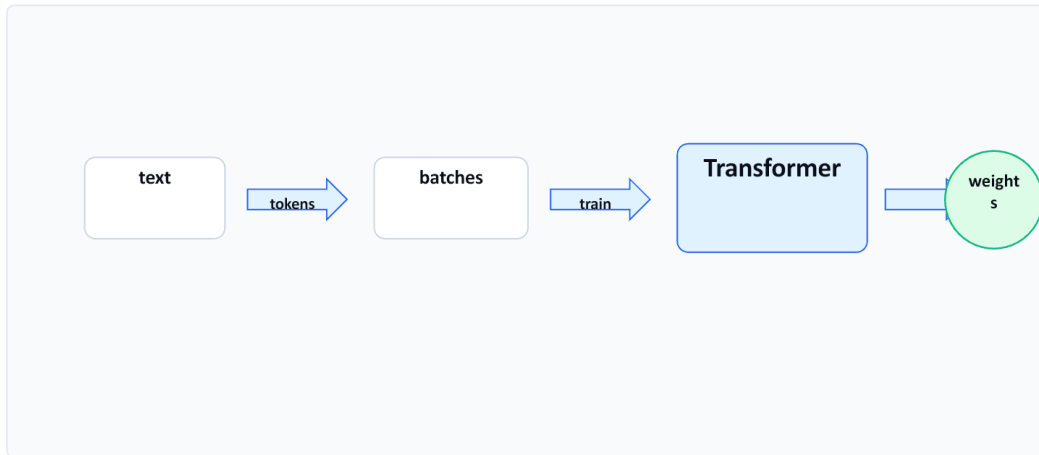
- GPT의 학습 목표는 단순합니다. 지금까지의 token을 보고 다음 token을 맞추는 것입니다.
- 예를 들어 'The capital of France is' 다음에 'Paris'가 오도록 확률을 높입니다.
- 이 단순한 목표를 엄청난 데이터와 큰 모델로 반복하면 다양한 언어 능력이 생깁니다.

출처 참고: Transformer 관련 내용은 Vaswani et al. (2017), GPT 관련 내용은 OpenAI GPT 계열 논문 및 Brown et al. (2020)에 기반합니다.

Slide 31. Pretraining compresses patterns into weights

Scale

Pretraining compresses patterns into weights



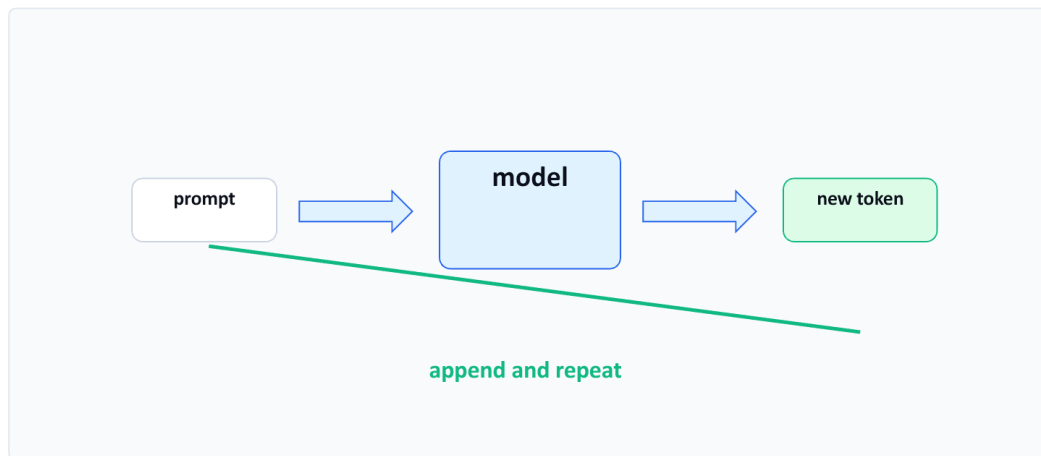
31

- Pretraining은 많은 텍스트를 token으로 바꾸고, next-token prediction을 반복해 weight에 패턴을 저장하는 과정입니다.
- 모델이 문법, 표현, 사실, 추론 패턴을 모두 명시적 규칙이 아니라 weight 안에 통계적으로 압축합니다.
- 이후 prompt를 넣으면 이 weight들이 다음 token 확률을 계산하는 데 사용됩니다.

Slide 32. Inference repeats the same move

Generation

Inference repeats the same move



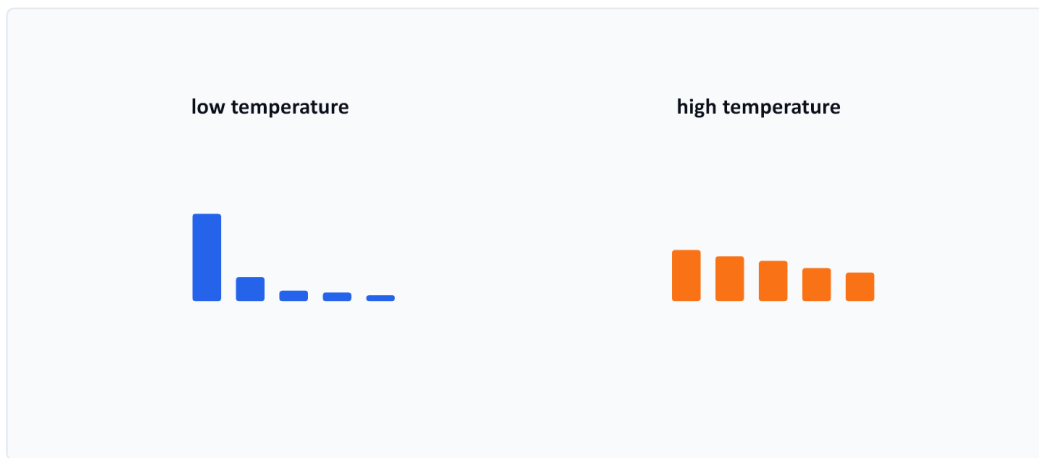
32

- Inference에서는 사용자의 prompt가 들어가고, 모델이 다음 token 하나를 예측합니다.
- 그 token을 문장 뒤에 붙인 뒤 다시 다음 token을 예측합니다.
- 우리가 보는 긴 답변은 이 작은 동작이 반복되어 만들어진 결과입니다.

Slide 33. Sampling changes the voice

Decoding

Sampling changes the voice



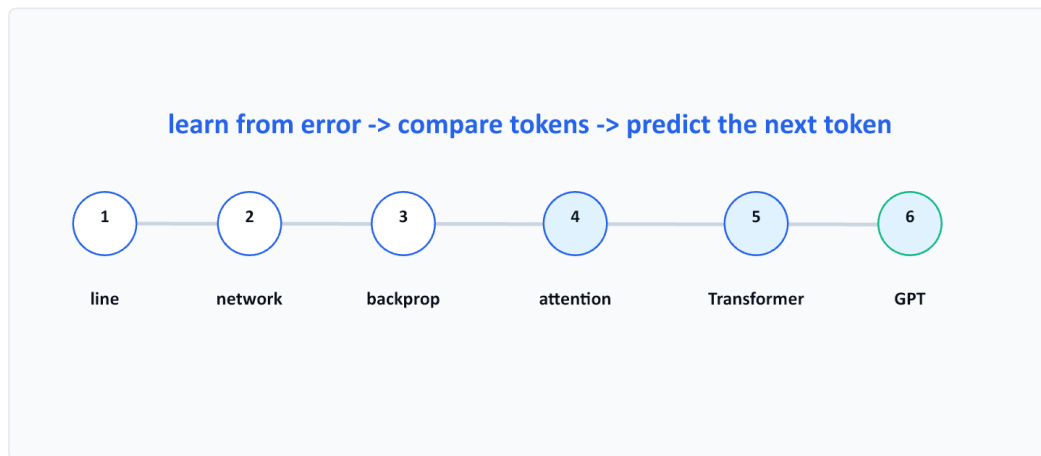
33

- 모델은 다음 token 하나만 정답으로 고르는 것이 아니라 여러 후보에 확률을 줍니다.
- Temperature가 낮으면 가장 가능성 높은 token을 더 안정적으로 고르고, 높으면 다양한 후보를 더 자주 선택합니다.
- 그래서 같은 모델도 decoding 설정에 따라 더 보수적이거나 더 창의적인 문체를 보일 수 있습니다.

Slide 34. The whole path is one learning story

Recap

The whole path is one learning story



34

- 마지막 장은 전체 내용을 하나로 묶습니다. 선형 회귀는 오차를 줄이는 가장 단순한 출발점입니다.
- 신경망과 backpropagation은 이 아이디어를 깊고 복잡한 함수로 확장합니다.
- Attention과 Transformer는 언어의 token 관계를 직접 비교하게 만들고, GPT는 이를 next-token prediction으로 대규모 학습한 모델입니다.

출처 참고: Transformer 관련 내용은 Vaswani et al. (2017), GPT 관련 내용은 OpenAI GPT 계열 논문 및 Brown et al. (2020)에 기반합니다.